

CS 440 Assignment 1 Report

Team Details

- Reuben Geronimo (`rg1090`)
- Orland Geronimo (`ogg9`)

Project Summary

In this assignment, we implemented and compared three repeated-planning methods for unknown gridworlds:

- Repeated Forward A*
- Repeated Backward A*
- Adaptive A*

Each maze is 101×101 , the agent starts at the top-left cell, and the goal is the bottom-right cell. The agent does **not** know the full map in advance. It only discovers blockage status as it moves, then replans when needed.

All comparison numbers in this report come from the same 50 mazes (seed `42`). We compared methods mostly using:

- found rate
- expanded nodes (primary metric)
- runtime in milliseconds
- replans

This report keeps the language simple but still gives complete answers for all required parts.

Part 1a - Why the first move is east in Figure 8

The agent plans with the freespace assumption, so unknown cells are treated as unblocked until observed otherwise. In Figure 8, the first search is done on that assumed map, and moving east is on a shortest presumed-unblocked path to the target.

If multiple first moves have the same cost (for example east and north), the implementation still has to pick one. That choice is determined by tie-breaking and neighbor processing order. Our implementation is deterministic in this situation, so it consistently chooses east for that example.

What this means: the east move is not because the agent already knows north is blocked; it is because under initial uncertainty and tie-handling, east is the selected shortest option.

Part 1b - Why the agent always terminates in finite time; move upper bound

Let U be the number of truly unblocked cells in the finite grid.

1. **Why it terminates in finite time**

- Each planning cycle is finite.
- After each cycle, the agent either reaches the goal, proves no presumed path exists, or learns new blockage info.
- Because the grid is finite, the amount of new information is finite.
- So the whole process must finish in finite time.

2. **Move bound**

- In one cycle, the agent follows a shortest presumed-unblocked path, so the path length is at most $U-1$.
- The number of meaningful replans is finite and bounded in finite grids.

- A coarse bound is:

$$\text{moves} \leq U \cdot (U-1) < U^2.$$

So the total number of moves until success/failure is upper-bounded by the number of unblocked cells squared.

Why this argument is convincing: nothing in the process can keep increasing forever in a finite grid. Each replan either finishes the problem or adds new knowledge, and there are only finitely many cells to learn about.

Part 2 - Effects of tie-breaking in Repeated Forward A*

We compared two versions:

- `max_g`: break `f` ties toward larger `g`
- `min_g`: break `f` ties toward smaller `g`

All runs were on the same 50 mazes (101x101, seed 42).

Observations

- Found rate: same for both (26/50)
- Expanded nodes:
 - Mean: 8,920 (`max_g`) vs 201,837 (`min_g`)
 - Median: 8,790 (`max_g`) vs 242,449 (`min_g`)
 - Per-maze: `max_g` better on 46/50, ties 4/50, `min_g` better on 0/50
- Runtime:
 - Mean: 76.7 ms (`max_g`) vs 1,190.4 ms (`min_g`)
 - Median: 80.9 ms (`max_g`) vs 1,445.2 ms (`min_g`)

Explanation

When `f = g + h` ties happen, choosing larger `g` tends to move search focus closer to the goal side of the same `f` contour. Choosing smaller `g` keeps more effort near the start side.

That difference becomes very large in repeated planning because each search happens in an evolving partial map. In our runs, `min_g` repeatedly paid a much larger expansion cost, so runtime grew by an order of magnitude.

Practical takeaway: for this project, forward A* with `max_g` is clearly the better tie strategy.

Part 3 - Repeated Forward A* vs Repeated Backward A*

We compared:

- Forward (`max_g`)
- Backward (`max_g`)

Observations

- Found rate: same (26/50)
- Expanded nodes:
 - Mean: 8,920 (forward) vs 102,180 (backward)
 - Median: 8,790 (forward) vs 101,170 (backward)
 - Forward better on expansions: 50/50 mazes
- Runtime:
 - Mean: 77.1 ms (forward) vs 1,325.6 ms (backward)
 - Median: 82.2 ms (forward) vs 1,283.9 ms (backward)
 - Forward better on runtime: 50/50 mazes

Explanation

In this problem, new information is discovered around the moving agent. Forward replanning starts exactly where new information matters most: the current agent position.

Backward replanning starts from the goal and plans toward a changing agent state. In our tests, that was consistently less efficient under partial observability, even though the success rate stayed the same.

Practical takeaway: both methods can solve the same mazes, but repeated forward A* is much cheaper here.

Part 4a - Why Manhattan distance is consistent here

Heuristic:

$$h(s) = |x_s - x_g| + |y_s - y_g|.$$

For neighboring cells s and s' in a 4-direction grid, one step changes Manhattan distance by at most 1, so:

$$h(s) \leq 1 + h(s').$$

Since each move cost is $c(s, s') = 1$, this is exactly:

$$h(s) \leq c(s, s') + h(s').$$

So Manhattan distance is consistent.

This is exactly the consistency condition A* needs. So Manhattan is a valid and safe heuristic choice for this 4-neighbor grid setup.

Part 4b - Why Adaptive A* keeps heuristics admissible/consistent when costs increase

Adaptive A* updates expanded states with:

$$h_{\text{new}}(s) = g(s_{\text{goal}}) - g(s).$$

At update time, that value is a valid lower bound to goal distance for expanded states, so it is admissible.

Later, edge costs can only increase (not decrease), which means true shortest distances cannot go down; therefore an admissible lower bound stays admissible.

For consistency: updates come from shortest-path g values in that search, which satisfy triangle-inequality-style relations. Non-updated states keep previously consistent values. With only cost increases and these updates, consistency is preserved across searches.

So Adaptive A* keeps initially consistent heuristics both admissible and consistent.

Why this matters in practice: Adaptive A* can safely increase heuristic values over time without breaking A* correctness.

Part 5 - Repeated Forward A* vs Adaptive A*

We compared:

- Forward (``max_g``)
- Adaptive (``max_g``)

Observations

- Found rate: same (``26/50``)
- Expanded nodes:
 - Mean: ``8,920`` (forward) vs ``8,549`` (adaptive)
 - Median: ``8,790`` (forward) vs ``8,717`` (adaptive)
 - Per-maze: adaptive better ``34``, ties ``14``, worse ``2``
- Runtime:
 - Mean: ``76.0 ms`` (forward) vs ``74.6 ms`` (adaptive)
 - Median: ``81.4 ms`` (forward) vs ``78.6 ms`` (adaptive)

Explanation

Adaptive A* reuses prior search work by raising heuristic values on expanded states:

```
\[
h_{new}(s)=g(s_{goal})-g(s).
\]
```

That usually focuses later replans and lowers expansions, which is exactly what we observed (adaptive won 34 mazes on expanded nodes).

Runtime improvement is smaller and less stable because:

- the absolute runtimes are already small,
- OS/runtime noise affects milliseconds more than expansion counts.

Practical takeaway: Adaptive A* is a good improvement over forward A* when the main goal is reducing search effort.

Part 6 - Statistical significance (how + results)

To test whether differences are real (not just sample noise), we use a paired two-sided sign test across the same 50 mazes.

For each maze:

```
\[
d_i = metric_{B,i} - metric_{A,i}.
\]
```

- ``B`` win if ``d_i < 0``
- ``B`` loss if ``d_i > 0``
- ties (``d_i=0``) dropped

Null hypothesis:

```
\[
H_0: P(\text{B win}) = 0.5.
\]
```

Two-sided exact p-values are computed from $\text{Binomial}(n, 0.5)$, where $n = \text{wins} + \text{losses}$.

Why we used a paired test: each algorithm is evaluated on the same maze set, so comparisons should be made maze-by-maze, not with independent-sample assumptions.

Why we use expanded nodes as the primary metric: it is more stable and less affected by machine load than runtime.

Expanded-node significance results

- Part 2 (`min\_g` vs `max\_g`): wins=0, losses=46, ties=4
 \(

\approx 2.84 \times 10^{-14}\)
- Part 3 (`bwd` vs `fwd`): wins=0, losses=50, ties=0
 \(

\approx 1.78 \times 10^{-15}\)
- Part 5 (`adaptive` vs `fwd`): wins=34, losses=2, ties=14
 \(

\approx 1.94 \times 10^{-8}\)

At \(\alpha=0.05\), all three expanded-node differences are statistically significant.

Runtime Interpretation

For Part 5 runtime, wins=25 and losses=25 (ties=0), so \(\approx 1.0\). That means no statistically significant paired runtime advantage, even though the mean/median runtime is slightly better for adaptive.

Final Conclusion

Across these 50 mazes:

- In Part 2, `max\_g` tie-breaking is clearly superior to `min\_g`.
- In Part 3, repeated forward A* is much more efficient than repeated backward A* in this setup.
- In Part 5, Adaptive A* gives a consistent expansion benefit over forward A*.

So the most practical overall choice in this project is to use forward-style replanning with strong tie-breaking (`max\_g`), and use Adaptive A* when you want better repeated-search efficiency over time.